pfi

Python for Fixed Income

COURSE 6 · NEURAL NETWORKS

Windows and Walk-Forward on the Yield Curve

WEEK 3 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The sequence model, framed

The question, in the past tense

For each date in the development years: how close to that date's 10 Yr change, in bp, could a model fitted only on earlier dates get, from the 20 changes before it?

Inside the file

Every fitted value is for a date in the file, from earlier dates only; nothing after its last date.

One baseline

The last value carried forward: for changes, a change of zero.

Not evidence

A walk-forward error describes past dates, not any later date.

Allowed names

fitted change in bp

walk-forward error

the last value carried forward

Never called

a forecast or a prediction

tomorrow's move or a call

an outlook or a view on rates

Development years and held-out years, set first



2015-01-02 to 2022-12-30: fit, score, choose

day 14, once

Why cut, when walk-forward is past-only

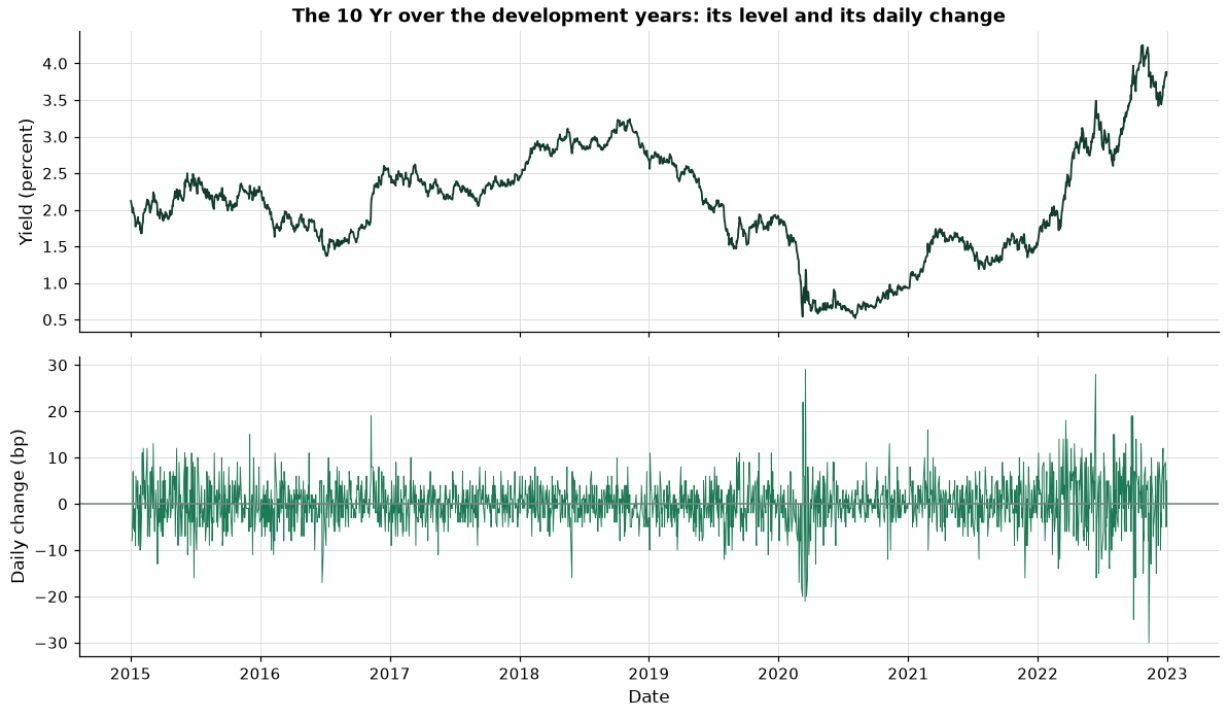
Every run of a walk-forward score is a chance to choose. Enough choices find a winner by luck; the held-out years are the dates nobody chose against.

Course 4 used these dates

Day 11: the same day's 30 Yr from the same day's 2 Yr and 10 Yr. Day 14: PCA. No row held out. Course 6 asks a new question, so it cuts its own.

Of the held-out years only the count and the first and last dates are read: 2023-01-03 to 2026-10-02.

The level wandered; the changes stay near zero



Mean absolute daily change 3.75 bp. A window of changes holds numbers that look alike across the years.

How windows work

1. The past

the rows just before a date

2. A window

20 rows, oldest first

3. Slide

one trading day along

4. The result

samples, steps, channels

You control

The window length and the channels.

What comes out

A 3-D array and one target per sample.

Windows: the days before each date



the window: the days before p

p

```
inputs = np.stack([rows[p - window:p] for p in positions])   # stops before p
```

(1980, 20, 1)

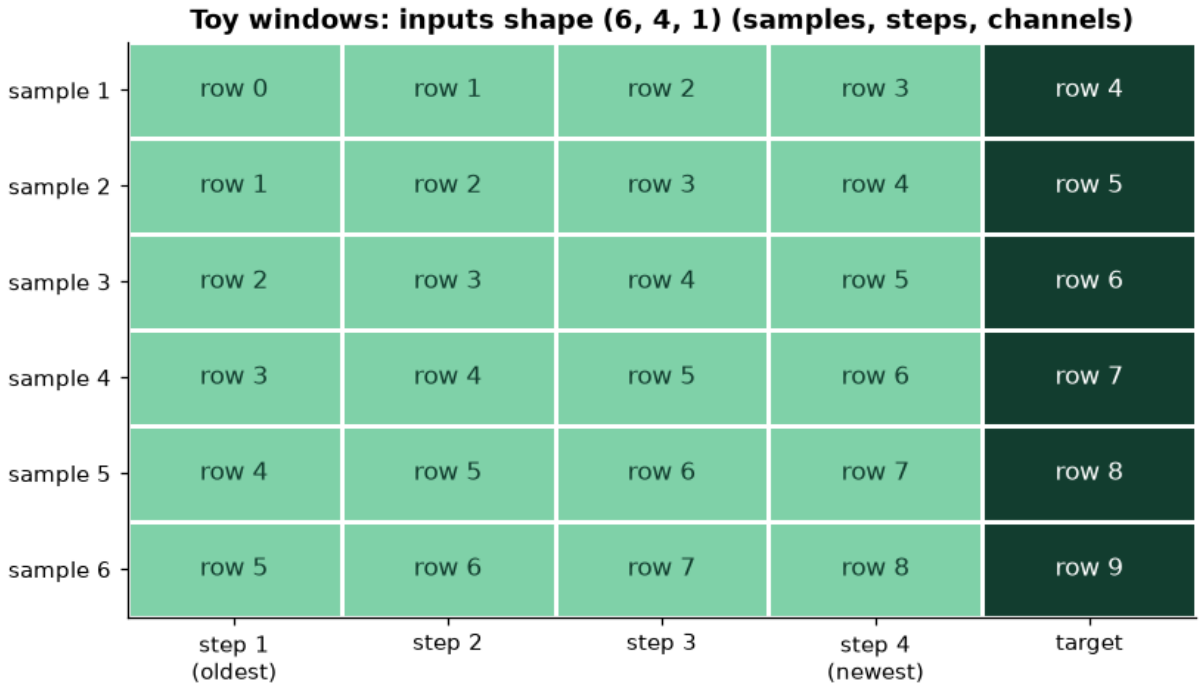
inputs: samples, steps, channels

2015-02-03

first target date; its window is the 20 trading days before it

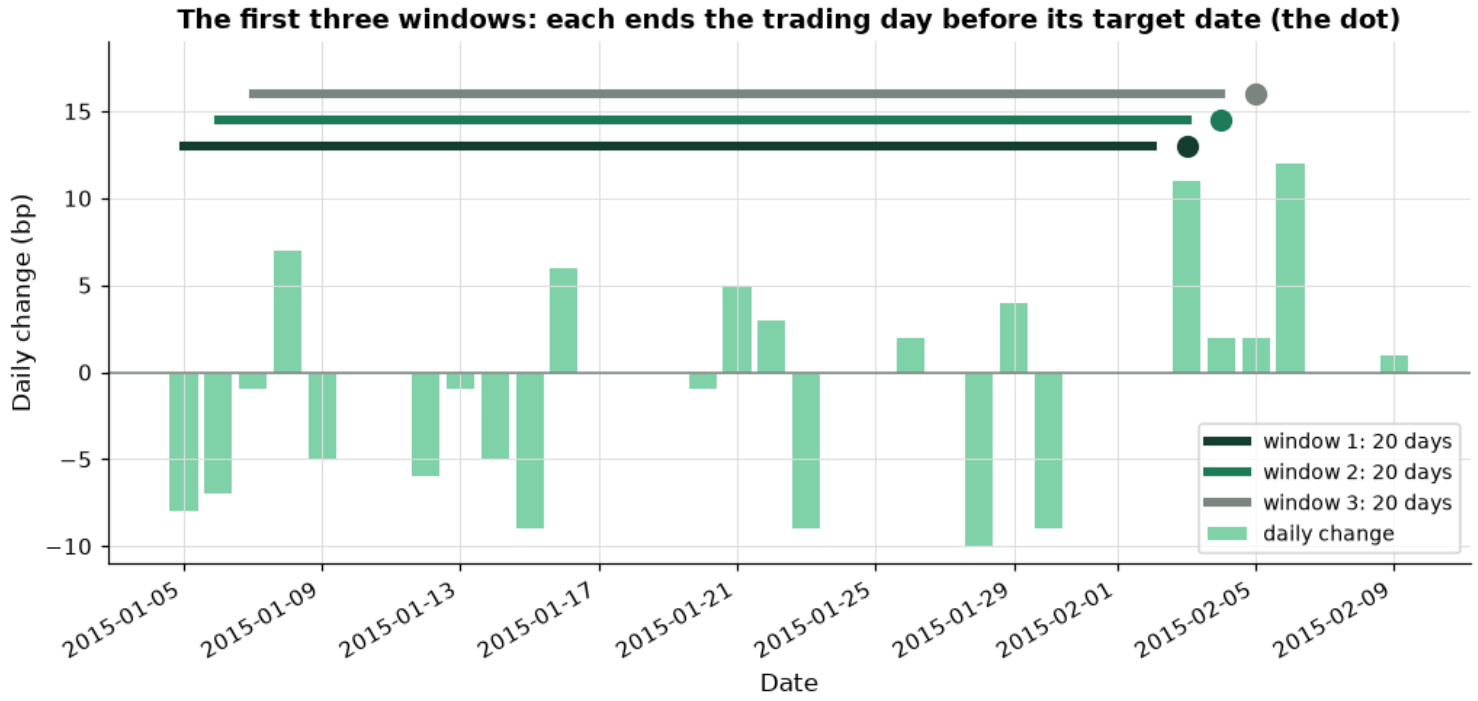
A window that held its own date's change would hand the model its answer.

Toy windows as a table



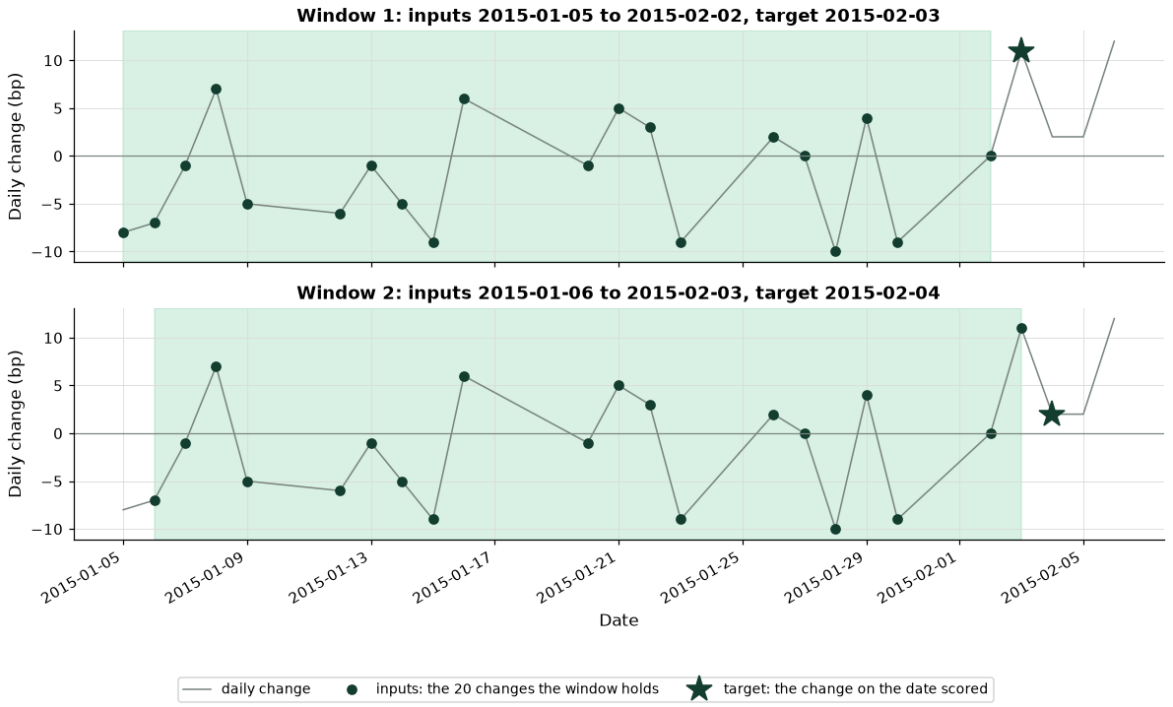
Each sample slides one row along; the target (dark) is the row just past the newest step. The array behind the table is shaped (6, 4, 1).

The first three windows, on the data



Each line stops one trading day short of its dot: window 1 ends on 2015-02-02 and is scored on 2015-02-03.

Inputs and target, marked



Window 2 slides one trading day: it drops the first day and adds window 1's target as its newest input.

How walk-forward scoring works

1. Fit

the first 504 target dates

2. Score

the next 252

3. Grow

add them to the training dates

4. Repeat

while a full block is left

You control

The first training block and the test block sizes.

What comes out

A list of folds, and one score per fold.

Two rules to beat, by fold

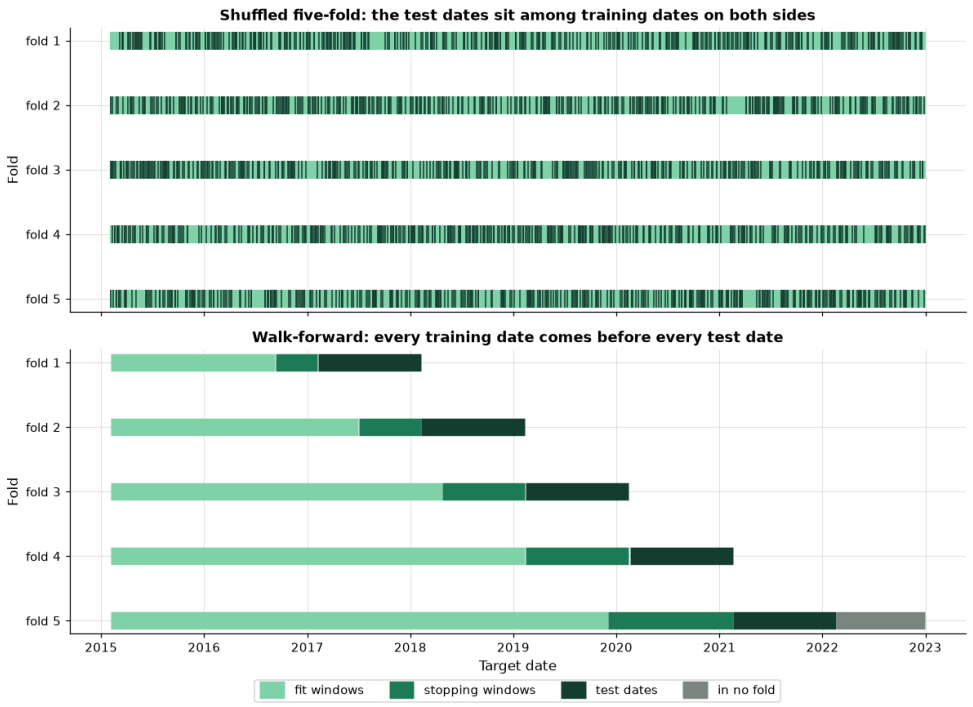
Fold	Test dates	Zero change	Training mean
fold 1	2017-02-07 to 2018-02-07	2.72	2.74
fold 2	2018-02-08 to 2019-02-12	2.76	2.79
fold 3	2019-02-13 to 2020-02-14	3.44	3.46
fold 4	2020-02-18 to 2021-02-18	3.53	3.53
fold 5	2021-02-19 to 2022-02-17	3.59	3.59

MAE in bp, mean of 5 folds

Zero change 3.21; training mean 3.22. Fitting one number did not beat fitting nothing.

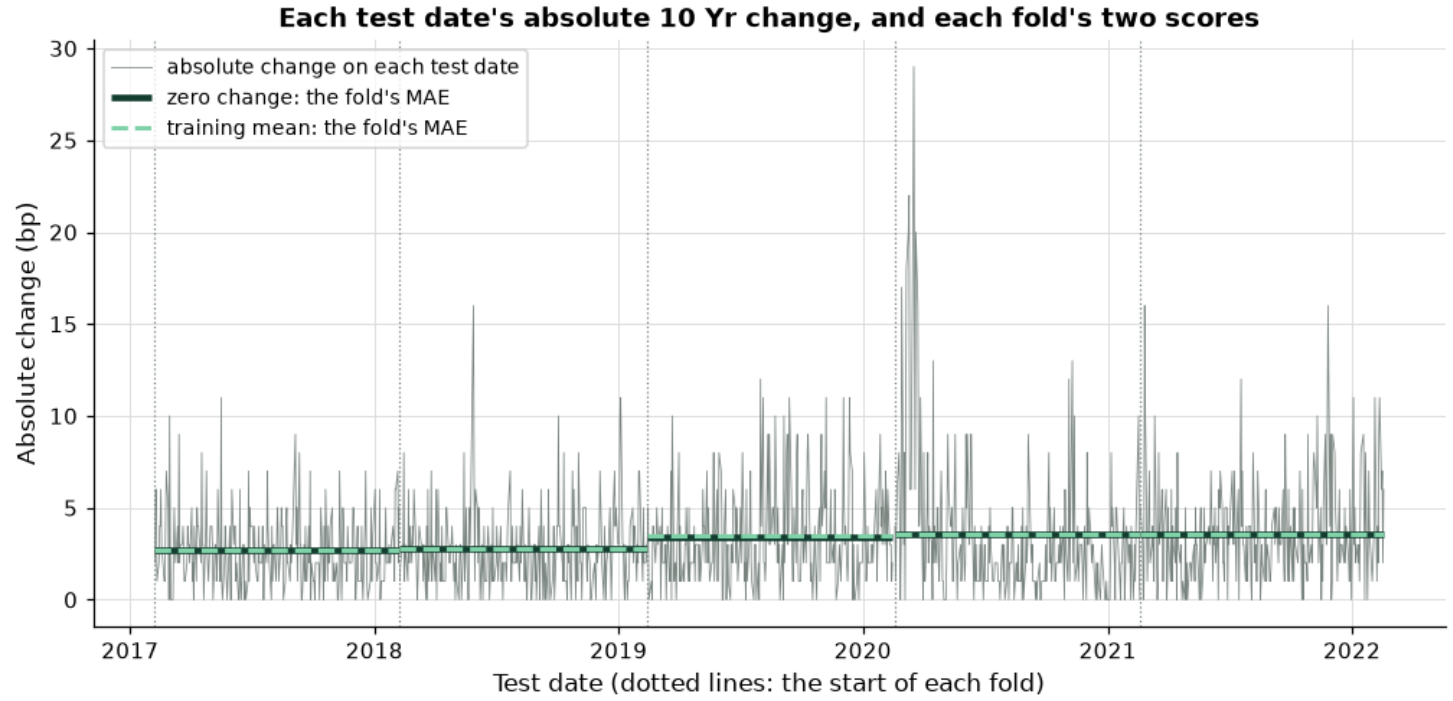
504 dates to fit, then 252 at a time. The last 216 dates, from 2022-02-18, fit no sixth year.

Walk-forward folds on a timeline



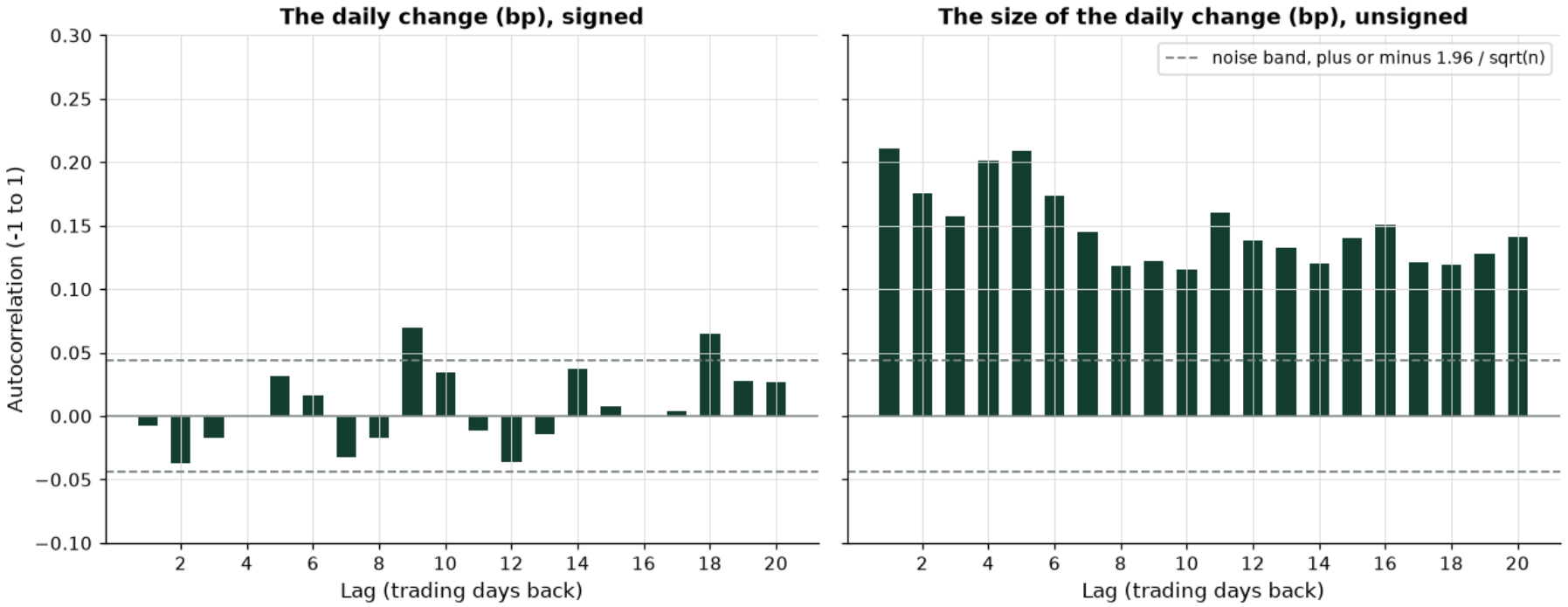
Walk-forward: every training date is before every test date. Shuffled: most test dates have a training date on both sides.

Zero change's score is the average size of the change



Zero change: 2.72 bp on fold 1 up to 3.59 bp on fold 5; the training mean sits almost on it.

What is in a window to find



The direction of earlier changes carries almost nothing to fit; the size of earlier changes carries a little about the size that follows.

How a fold of a network works

1. Windows

cut from the training dates

2. Fit

a fresh network on four fifths

3. Stop

on the last fifth

4. Score

the test dates that follow

You control

The build function, the window, and the block sizes.

What comes out

One row per fold; fitted changes in bp.

One fold by hand: four doors closed

```
Flatten -> Dense(16, relu) -> Dense(1): 353 weights and biases, fitted on the standardized change
```

1. Scaler

fitted on the rows inside the training windows only

2. Target

standardized on the training dates, back to bp at the end

3. Stopping rows

the last fifth of the training windows, in date order

4. Windows

each ends the day before its date

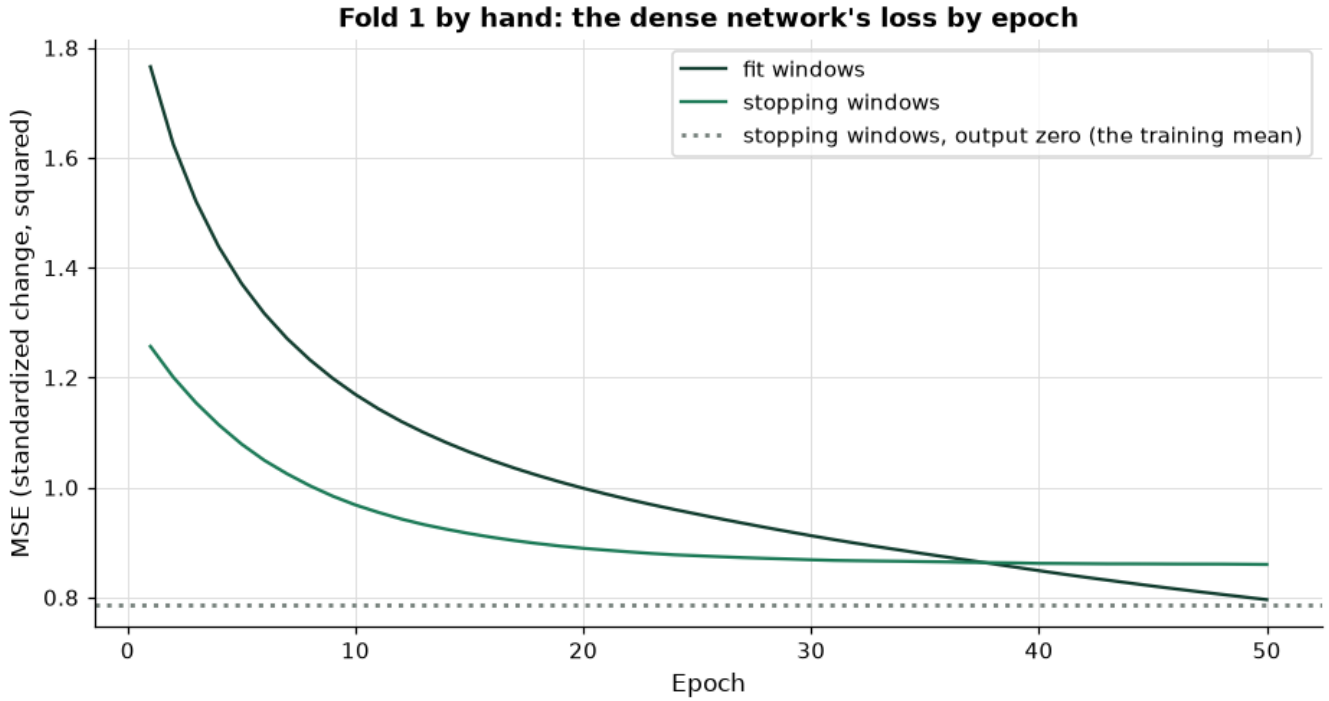
2.87 bp

fold 1, network: 404 windows fitted, 100 to stop

2.72 bp

fold 1, zero change, same dates

Fold 1's loss by epoch



Both fall for 50 epochs; the stopping curve never gets below the dotted line, the training mean's loss.

walk_forward_network over five folds

Fold	Network	Zero change	Training mean	Best epoch
fold 1	2.87	2.72	2.74	50
fold 2	2.90	2.76	2.79	28
fold 3	3.64	3.44	3.46	25
fold 4	3.70	3.53	3.53	11
fold 5	3.62	3.59	3.59	8

3.35 bp

dense network, mean of 5 folds

3.21 bp

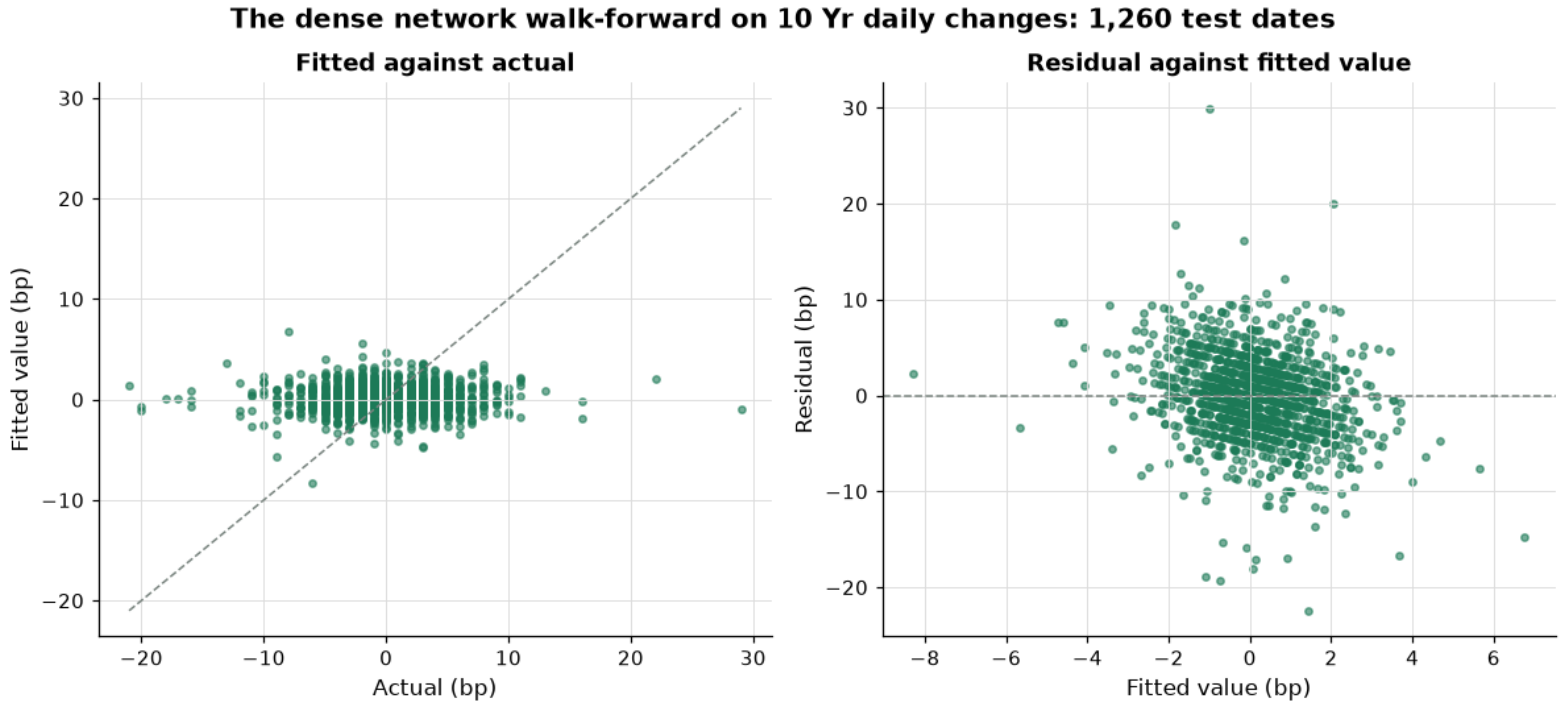
zero change, mean of 5 folds

0 of 5

folds below zero change

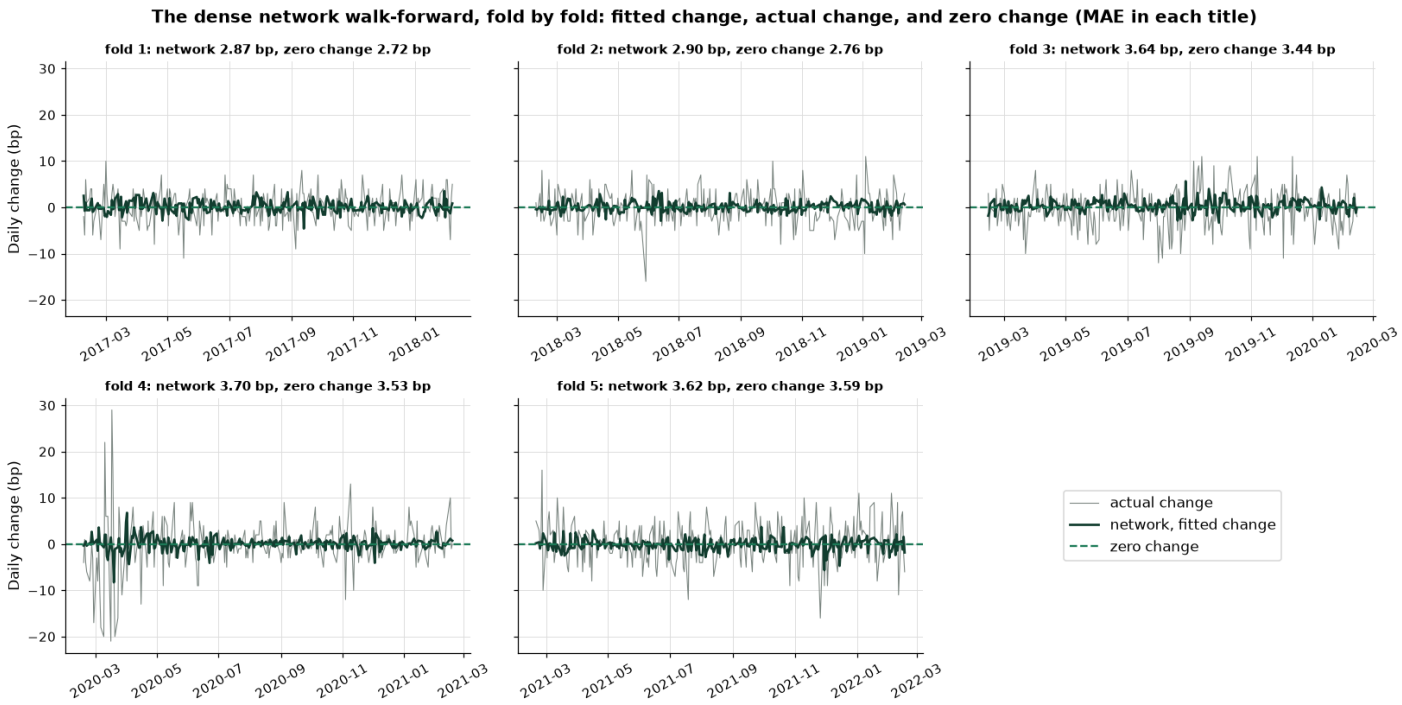
The 20 changes before a date held nothing this network could fit that beat the last value carried forward. Reported, not tuned away.

Fitted change against actual change



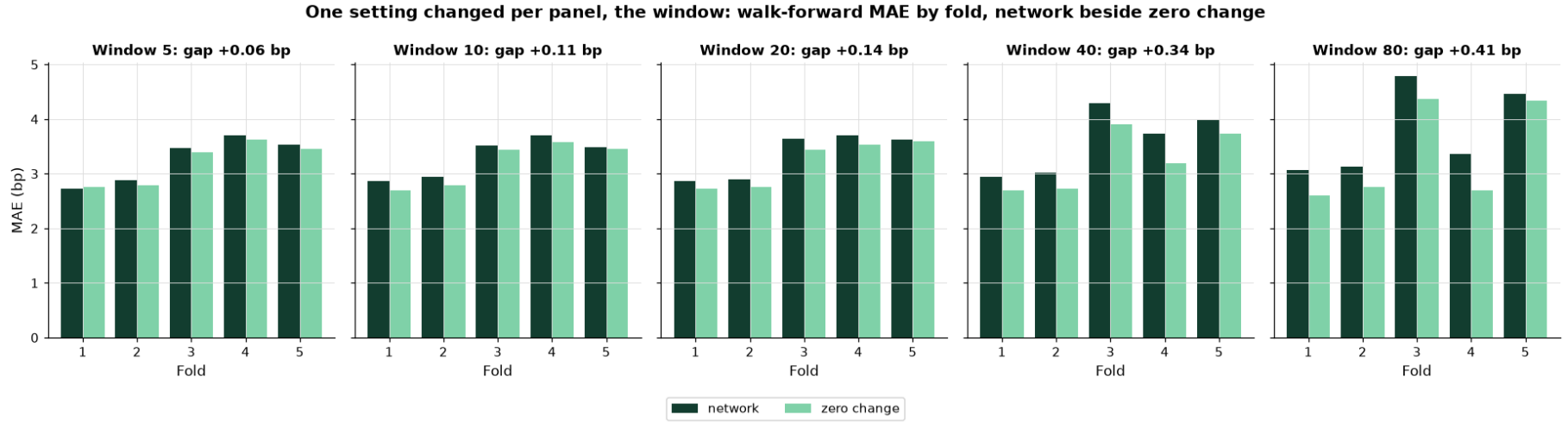
Standard deviation: actual 4.33 bp, fitted 1.31 bp. A flat band, not the 45-degree line.

Fitted against actual, fold by fold



In every fold the fitted line stays near zero while the actual change swings; zero change scores lower in all five.

The window, one setting changed per panel



Window	Numbers	Network	Zero change	Gap
5	113	3.26	3.20	0.06
10	193	3.30	3.19	0.11
20	353	3.35	3.21	0.14
40	673	3.59	3.25	0.34
80	1,313	3.76	3.35	0.41

Levels instead of changes

Fold	Network	Last value	Best epoch	Outside range
fold 1	3.69	2.72	50	16
fold 2	4.22	2.76	1	177
fold 3	10.77	3.44	3	0
fold 4	6.02	3.53	50	247
fold 5	11.16	3.59	1	0

7.17 bp

network on levels

3.21 bp

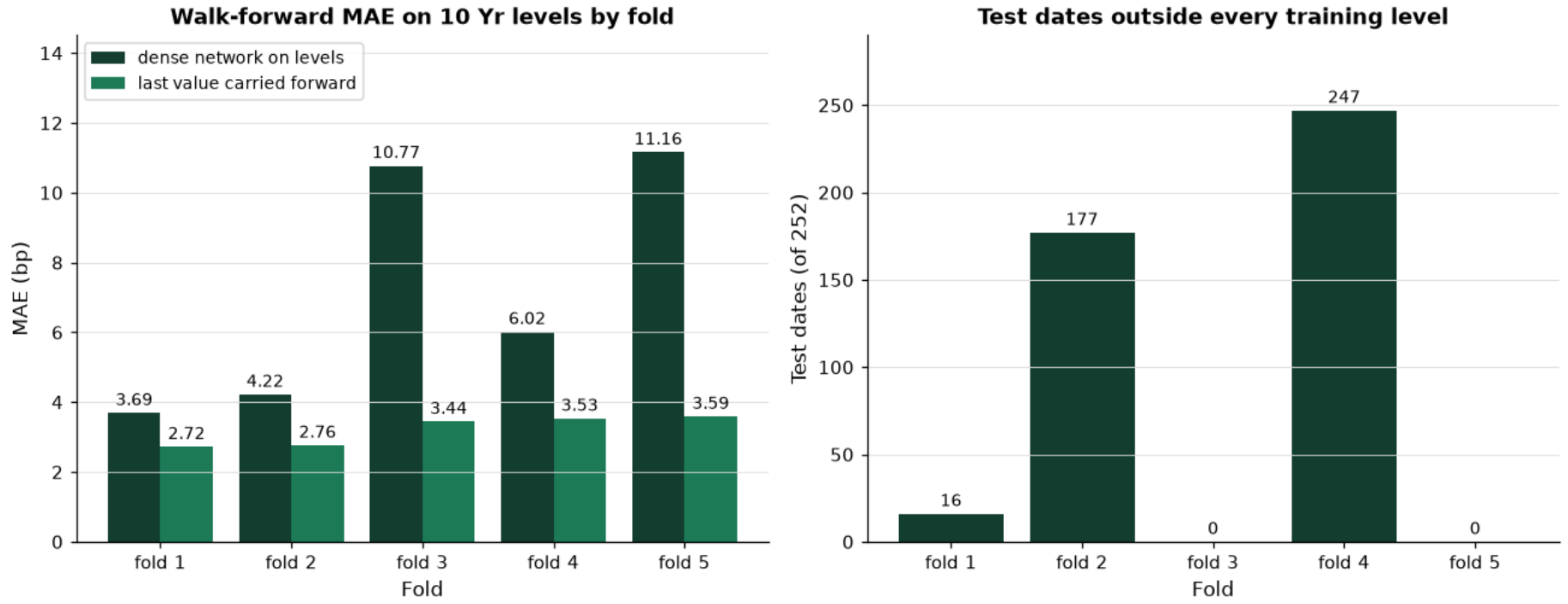
last value carried forward

Course 4's reason, checked

Most levels outside the training range was not the worst fold.

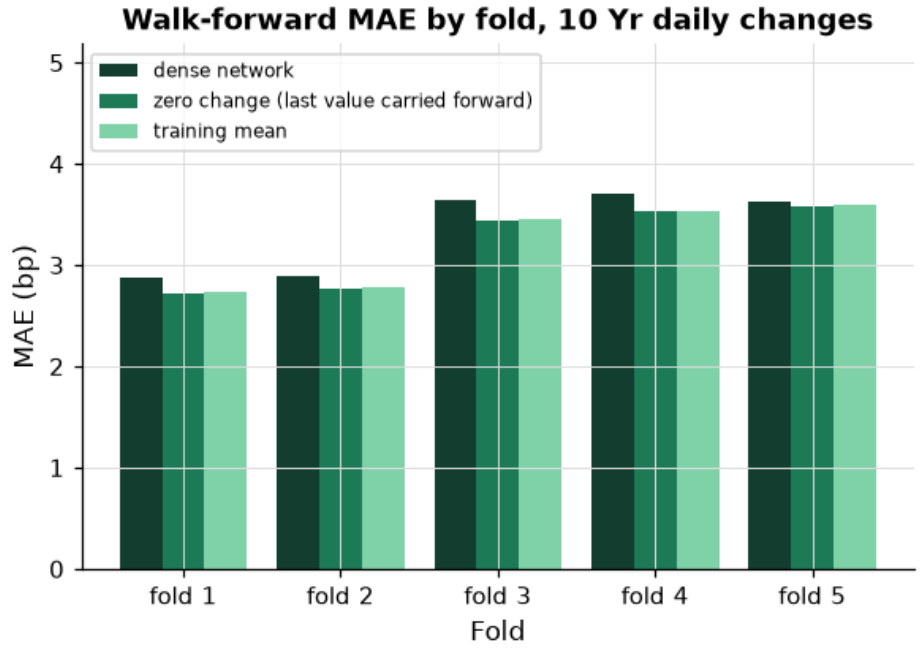
The last value gets the copy for free. The network has to learn it, and did not on any fold.

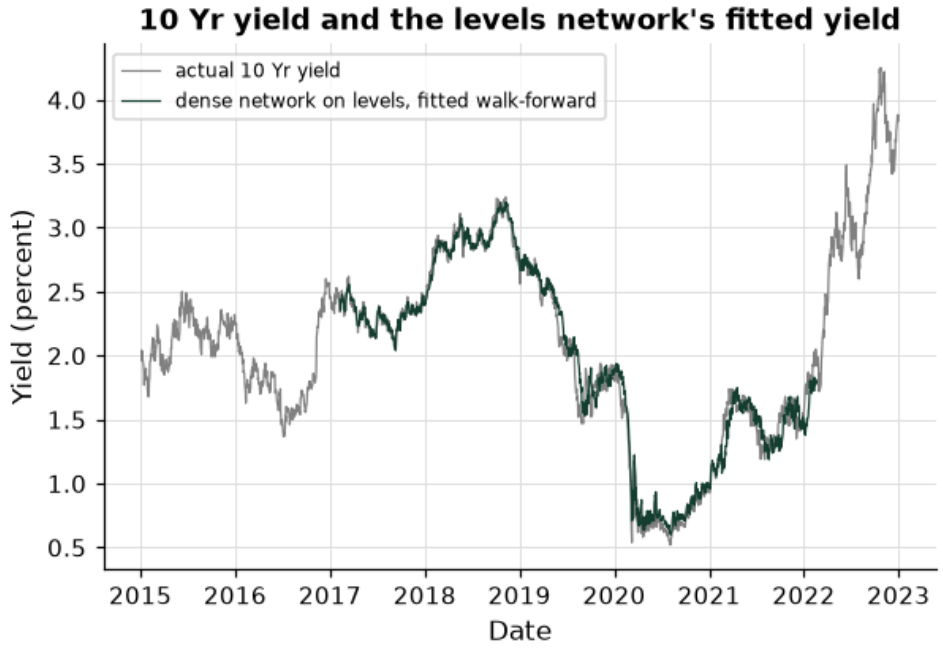
Levels by fold: the two panels do not line up



The two worst folds on the left have no test date outside the training range on the right.

Fold by fold





All three rise in folds 3 to 5: the 10 Yr moved more on an average day then. The rules did not change.

What this shows, and what it does not

What it shows

On 1,260 past dates the network was further from the actual change than zero change, on every fold.

What it does not

That no model could do better; or anything about a date outside the folds.

Why report it

Each change tried is a choice on these folds. Enough choices find a winner by luck.

A walk-forward error on past dates is not evidence about any later date, and not a view on rates.

Excel side by side

Excel side by side (tested in Excel)

`=(B3-B2)*100`	change in bp, C3 filled down
`=AVERAGE(OFFSET(C23,-20,0,20,1))`	the window's mean; first -2.2000
`=INDEX(OFFSET(C23,-20,0,20,1),20)`	the window's last day: the row above
`=AVERAGE(ABS(curve!C527:C778))`	zero change, fold 1: 2.7222
`=AVERAGE(ABS(curve!C527:C778-AVERAGE(curve!C23:C526)))`	training mean: 2.7362
`=AVERAGE(ABS(curve!C527:C778-curve!F527:F778))`	network: 2.8715

No walk-forward routine and no network training in Excel. Every number above equals Python's within 1e-12.

Save the day: git blame and the sequence log

```
git blame -s --root -L "/def fit_arrays/,+1" -L "/def make_windows/,+1" netkit.py
e85c9086 110) def fit_arrays(transformer, X_fit, *others) -> tuple:
3d633dc1 574) def make_windows(values, target, window: int, positions)
```

Two hashes: fit_arrays came in the morning commit, make_windows in today's. A pattern keeps working when lines above it move.

```
model,channels,window,rows,folds,mae_bp,zero_change_mae_bp,folds_below_zero_change
zero_change,1,20,development,5,3.2087,3.2087,0
training_mean,1,20,development,5,3.2216,3.2087,0
dense_16_changes,1,20,development,5,3.3472,3.2087,0
dense_16_levels,1,20,development,5,7.1715,3.2087,0
```

The sequence log: one row per rule or network, on the development years' folds.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows stop, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: windows end on the date they are scored on: fold 1 MAE 0.84 bp against 2.87 bp; the test flags 1,912 windows.

Code that runs is not the same as code that is right.

What you can do now

- State a sequence question in the past tense
- Cut development and held-out years first
- Build windows that end before their date
- Score zero change and the training mean by fold
- Fit walk-forward folds with nothing leaking in

Week 3, Day 2 Recurrent networks: LSTM and GRU, on the same folds